

OptiCrop: Smart Agricultural Production Optimization Engine

Project Description:

OptiCrop: Smart Agricultural Production Optimization Engine is a machine learning-based application developed using Google Colab to help farmers make informed crop selection decisions based on soil and environmental conditions. The system analyzes important agricultural parameters such as Nitrogen (N), Phosphorus (P), Potassium (K), temperature, humidity, pH, and rainfall to recommend the most suitable crop for cultivation.

The machine learning model is built, trained, and tested in Google Colab using Python libraries such as NumPy, Pandas, Scikit-learn, Matplotlib, and Seaborn. Google Colab provides a cloud-based environment for data preprocessing, model training, visualization, and performance evaluation without requiring high-end local hardware.

The application helps farmers improve crop yield, optimize the use of fertilizers and water, and support sustainable farming practices through accurate, data-driven recommendations. Researchers and policymakers can also use the system to analyze crop-environment relationships and make informed agricultural decisions. Overall, OptiCrop provides an efficient, reliable, and intelligent solution for modern agriculture using machine learning and cloud-based development with Google Colab.

Scenarios :

Scenario 1: Smart Crop Recommendation

A farmer enters soil parameters such as Nitrogen (N), Phosphorus (P), Potassium (K), pH, temperature, humidity, and rainfall into the system. The machine learning model analyzes the data and recommends the most suitable crop to maximize yield and improve farming efficiency.

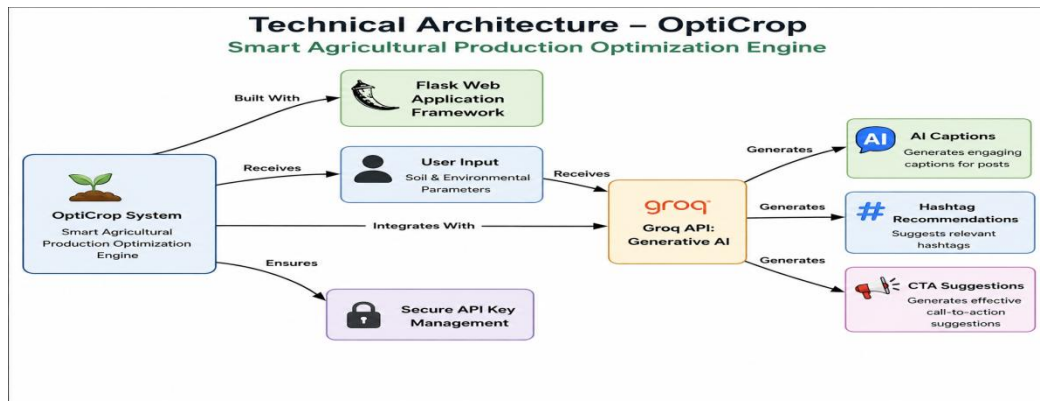
Scenario 2: Crop Suitability Analysis

A user wants to know whether the current soil and climate conditions are suitable for growing a specific crop. The application evaluates the input parameters and provides a crop suitability assessment along with productivity insights.

Scenario 3: Agricultural Research & Planning

An agricultural researcher or policymaker uses the system to analyze crop-environment relationships based on historical soil and weather data. The insights help in planning sustainable farming strategies, improving resource management, and supporting data-driven agricultural decisions.

Technical Architecture:



Description:

The OptiCrop: Smart Agricultural Production Optimization Engine follows a machine learning-based architecture that collects agricultural data, processes it, predicts the most suitable crop, and presents the results through a simple web interface. The system is designed to help farmers make accurate and data-driven crop selection decisions.

(Note: If you are using database in your project definitely you need to add ER Diagram along with description)

Pre-requisites:

Before running the application, ensure the following software and libraries are installed on your system.

Hardware Requirements

- Laptop/Desktop
- Minimum 8 GB RAM
- Intel Core i5 / AMD Ryzen 5 Processor or above
- At least 10 GB free storage
- Stable Internet connection

Software Requirements

- Google Colab
- Python 3.x
- Jupyter Notebook (Optional)
- Visual Studio Code (Optional, for Flask UI)
- Git & GitHub (Optional)

Python Libraries

- NumPy
- Pandas

- Scikit-learn
- Matplotlib
- Seaborn
- Flask

Dataset Requirements

- Crop Recommendation Dataset (CSV format)
- Features:
 - Nitrogen (N)
 - Phosphorus (P)
 - Potassium (K)
 - Temperature
 - Humidity
 - pH
 - Rainfall
- Target:
 - Crop Name

Project Workflow:

Milestone 1: Project Planning & Dataset Collection

- **Activity 1.1:** Problem Identification
- **Activity 1.2:** Crop Recommendation Dataset Collection
- **Activity 1.3:** Solution Architecture Design
- **Activity 1.4:** Development Environment Setup (Google Colab & Python Libraries)

Milestone 2: Data Preprocessing

- **Activity 2.1:** Data Cleaning
- **Activity 2.2:** Handling Missing Values
- **Activity 2.3:** Exploratory Data Analysis (EDA)
- **Activity 2.4:** Feature Selection & Data Splitting

Milestone 3: Machine Learning Model Development

- **Activity 3.1:** Model Training
- **Activity 3.2:** Model Evaluation
- **Activity 3.3:** Model Comparison
- **Activity 3.4:** Final Model Selection & Model Saving

Milestone 4: Crop Prediction System Development

- **Activity 4.1:** User Input Interface Development
- **Activity 4.2:** Prediction Module Integration
- **Activity 4.3:** Crop Recommendation Generation
- **Activity 4.4:** Result Display & Visualization

Milestone 5: Testing & Deployment

- **Activity 5.1:** Functional Testing
- **Activity 5.2:** Performance Evaluation
- **Activity 5.3:** GitHub Repository & Project Documentation

Milestone 1: Project Planning & Dataset Collection

The first milestone focused on understanding the project objectives, collecting the crop recommendation dataset, designing the solution architecture, and setting up the development environment. Proper planning established a strong foundation for implementing the machine learning model and developing the crop recommendation system efficiently.

Activity 1.1: Problem Identification:

The objective of this project is to develop an intelligent crop recommendation system that suggests the most suitable crop based on soil nutrients and environmental conditions. Farmers often face challenges in selecting the right crop due to changing climate conditions and limited knowledge of soil properties, which can reduce crop yield and resource efficiency. By leveraging Machine Learning algorithms, the proposed system provides accurate, reliable, and data-driven crop recommendations, helping farmers improve productivity and promote sustainable agricultural practices.

Activity 1.2: Dataset Collection:

The dataset for this project was collected from **Kaggle**. It contains soil nutrient information, environmental parameters, and crop labels used to train and evaluate the Machine Learning model.

The dataset includes the following important attributes:

- Nitrogen (N)
- Phosphorus (P)
- Potassium (K)
- Temperature
- Humidity
- pH
- Rainfall
- Crop Label

These features were preprocessed and used to build the crop recommendation model.

Dataset Source:

<https://www.kaggle.com/datasets/atharvaingle/crop-recommendation-dataset>

Activity 1.3: Solution Architecture Design:

A solution architecture was designed before implementation to define the complete workflow of the system. The architecture illustrates how soil and environmental data flow through different stages, including data collection, data preprocessing, feature engineering, machine learning model training, model evaluation, prediction generation, and result visualization.

The architecture provides a clear understanding of the interaction between different components of the system and ensures a modular, scalable, and maintainable project design.

Activity 1.4: Development Environment Setup

The following tools and technologies were configured before starting the implementation:

Software / Tool	Purpose
Google Colab	Machine Learning Model Development
Python 3.x	Programming Language
Pandas	Data Manipulation
NumPy	Numerical Computation
Scikit-learn	Machine Learning
Matplotlib	Data Visualization
Seaborn	Statistical Data Visualization
GitHub	Version Control & Project Repository
CSV Dataset	Crop Recommendation Dataset

Milestone 2: Data Preprocessing

The second milestone focused on preparing the crop recommendation dataset for Machine Learning model development using **Google Colab**. Raw agricultural datasets may contain missing values, inconsistent records, and unnecessary information that can affect prediction accuracy. Therefore, several preprocessing techniques were applied to improve the quality of the dataset before training the Machine Learning models. Proper preprocessing ensures that the model receives clean, consistent, and meaningful input data, leading to better crop recommendation performance.

Activity 2.1: Data Cleaning

The collected crop recommendation dataset was cleaned to remove inconsistencies and prepare it for analysis. The following preprocessing tasks were performed:

- Loaded the dataset into **Google Colab** using Pandas.
- Checked the dataset structure, rows, and columns.
- Verified data types of all features.
- Removed duplicate records (if any).
- Ensured that all input values were consistent and valid.

```

In [10]: import pandas as pd

df = pd.read_csv("/content/Crop_recommendation.csv")

df.head()

```

	N	P	K	temperature	humidity	ph	rainfall	label
0	90	42	43	20.879744	82.002744	6.502985	202.935536	rice
1	85	58	41	21.770462	80.319644	7.038096	226.655537	rice
2	60	55	44	23.004459	82.320763	7.840207	263.964248	rice
3	74	35	40	26.491096	80.158363	6.980401	242.864034	rice
4	78	42	42	20.130175	81.604873	7.628473	262.717340	rice

This process ensured that the dataset was clean and ready for further analysis.

Activity 2.2: Handling Missing Values

The dataset was examined for missing or null values to ensure data completeness before model training. The following steps were carried out:

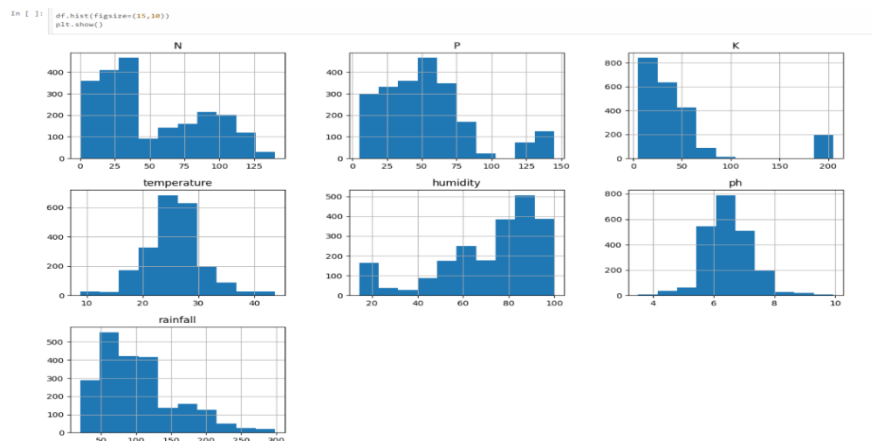
- Checked for missing values using Pandas.
- Verified that all features contained valid data.
- Filled or removed missing values whenever necessary.
- Confirmed that the final dataset contained no null values.

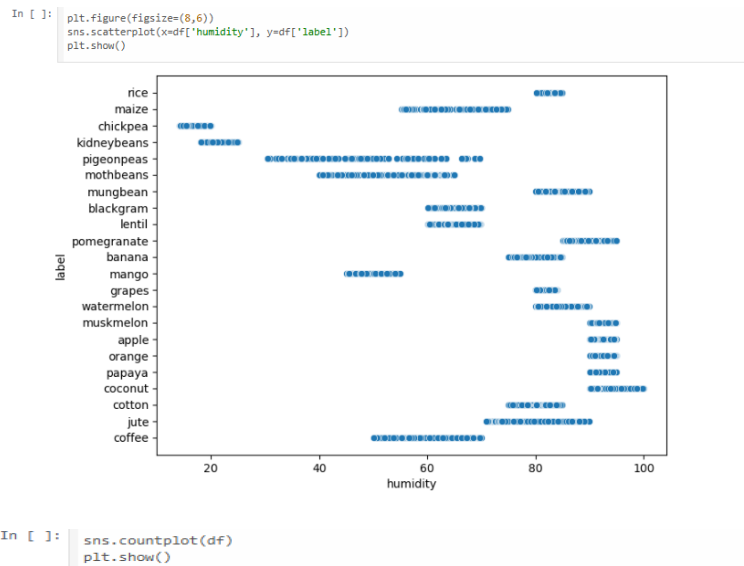
This helped improve the reliability and accuracy of the Machine Learning model.

Activity 2.3: Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) was performed to understand the distribution of agricultural features and their relationship with crop recommendations. The following analyses were carried out:

- Generated descriptive statistics for all features.
- Visualized feature distributions using histograms and box plots.
- Created correlation heatmaps to study relationships between variables.
- Analyzed the distribution of crop categories using count plots.





The visualizations were created using Matplotlib and Seaborn in Google Colab, providing valuable insights into the dataset before model development.

Activity 2.4: Feature Selection & Data Splitting

Relevant input features were selected to build the crop recommendation model. The selected features included Nitrogen (N), Phosphorus (P), Potassium (K), Temperature, Humidity, pH, and Rainfall, while the Crop Label was used as the target variable.

Milestone 3: Machine Learning Model Development

The third milestone focused on developing a Machine Learning model to recommend the most suitable crop based on soil nutrients and environmental conditions. The preprocessed dataset was used to train and evaluate different Machine Learning algorithms in **Google Colab**. The performance of each model was compared, and the best-performing model was selected for crop prediction.

Activity 3.1: Model Training

The preprocessed dataset was divided into training and testing datasets using **train_test_split()**. Two Machine Learning algorithms were trained using the training dataset.

The following algorithms were used:

- **K-Means Clustering**
- **Logistic Regression**

The models learned the relationship between soil parameters and crop labels to generate crop recommendations.

```
In [4]: from sklearn.model_selection import train_test_split

y = df["label"]
X = df.drop("label", axis=1)

print("Shape of x", X.shape)
print("Shape of y", y.shape)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0
)

print("The shape of X train", X_train.shape)
print("The shape of X test", X_test.shape)
print("The shape of y train", y_train.shape)
print("The shape of y test", y_test.shape)

Shape of x (2200, 7)
Shape of y (2200,)
The shape of X train (1760, 7)
The shape of X test (440, 7)
The shape of y train (1760,)
The shape of y test (440,)
```

Activity 3.2: Model Evaluation

The trained models were evaluated using the testing dataset to measure their prediction performance.

The following evaluation metrics were used:

- Accuracy Score
- Confusion Matrix
- Classification Report
- Precision
- Recall
- F1-Score

```
In [6]: from sklearn.metrics import classification_report

cr = classification_report(y_test, y_pred)
print(cr)
```

	precision	recall	f1-score	support
apple	1.00	1.00	1.00	18
banana	1.00	1.00	1.00	18
blackgram	0.88	1.00	0.94	22
chickpea	1.00	1.00	1.00	23
coconut	1.00	1.00	1.00	15
coffee	1.00	1.00	1.00	17
cotton	0.89	1.00	0.94	16
grapes	1.00	1.00	1.00	18
jute	0.87	0.95	0.91	21
kidneybeans	1.00	1.00	1.00	20
lentil	1.00	0.94	0.97	17
maize	1.00	0.89	0.94	18
mango	1.00	1.00	1.00	21
mothbeans	0.96	0.92	0.94	25
mungbean	1.00	1.00	1.00	17
muskmelon	1.00	1.00	1.00	23
orange	1.00	1.00	1.00	23
papaya	1.00	0.95	0.98	21
pigeonpeas	1.00	1.00	1.00	22
pomegranate	1.00	1.00	1.00	23
rice	0.96	0.88	0.92	25
watermelon	1.00	1.00	1.00	17
accuracy			0.98	440
macro avg	0.98	0.98	0.98	440
weighted avg	0.98	0.98	0.98	440

These metrics helped determine how accurately the models recommended the appropriate crop.

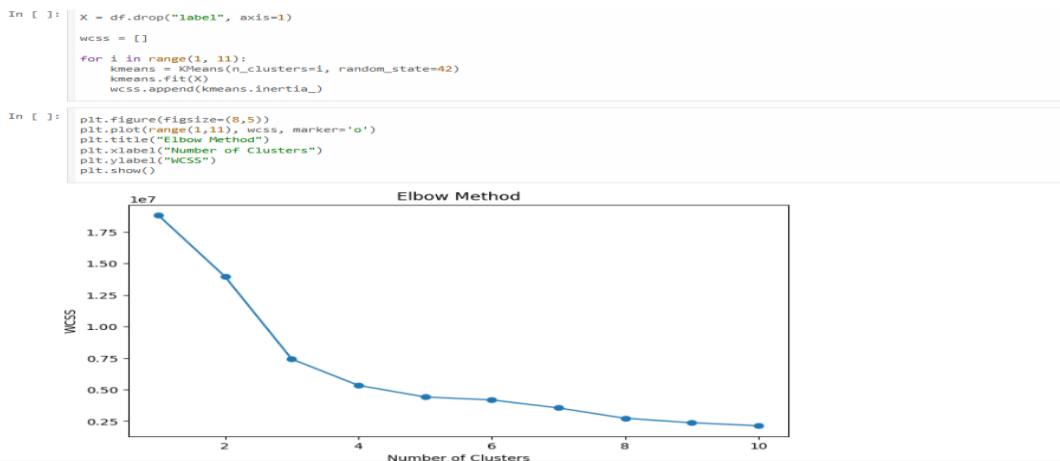
Activity 3.3: Model Comparison

The performance of **K-Means Clustering** and **Logistic Regression** was compared based on prediction accuracy and overall performance.

The comparison considered:

- Accuracy
- Precision
- Recall
- F1-Score
- Prediction Performance

The algorithm with better performance was selected for the final crop recommendation system.



Activity 3.4: Final Model Selection & Model Saving

Based on the evaluation results, the best-performing algorithm was selected as the final prediction model. The trained model was saved using the **Joblib** library as a **.pkl** file for future predictions.

The following tasks were completed:

- Selected the best-performing model.
- Saved the trained model (`crop_recommendation_model.pkl`).
- Verified that the model could be loaded successfully for prediction.

Screenshots to Include:

- `joblib.dump()` code
- Model saved successfully
- `joblib.load()` output

```
In [7]: import pickle

        pickle.dump(model, open("model.pkl", "wb"))

        print("Model saved successfully!")

Model saved successfully!
```

Milestone 4: Crop Prediction System Development

The fourth milestone focused on developing the crop prediction system by creating a user-friendly interface, integrating the trained Machine Learning model, generating crop recommendations, and displaying the prediction results. The system allows users to enter soil and environmental parameters and receive the most suitable crop recommendation instantly.

Activity 4.1: User Input Interface Development

A simple and interactive user interface was developed to collect agricultural parameters from the user. The interface allows users to enter the required input values for crop prediction.

The following input parameters are collected:

- Nitrogen (N)
- Phosphorus (P)
- Potassium (K)
- Temperature
- Humidity
- pH
- Rainfall

The interface was designed to ensure accurate data entry and improve user experience.

Activity 4.2: Prediction Module Integration

The trained **Logistic Regression** model was integrated with the crop prediction system. After the user enters the required input values, the application processes the data and passes it to the trained model. The prediction module analyzes the input parameters and generates the most suitable crop recommendation.

This integration enables the application to provide real-time predictions efficiently.

Activity 4.3: Crop Recommendation Generation

Once the input data is processed, the Machine Learning model predicts the most appropriate crop based on soil nutrients and environmental conditions. The recommendation is generated using the trained model and helps farmers select crops that are best suited for their field conditions.

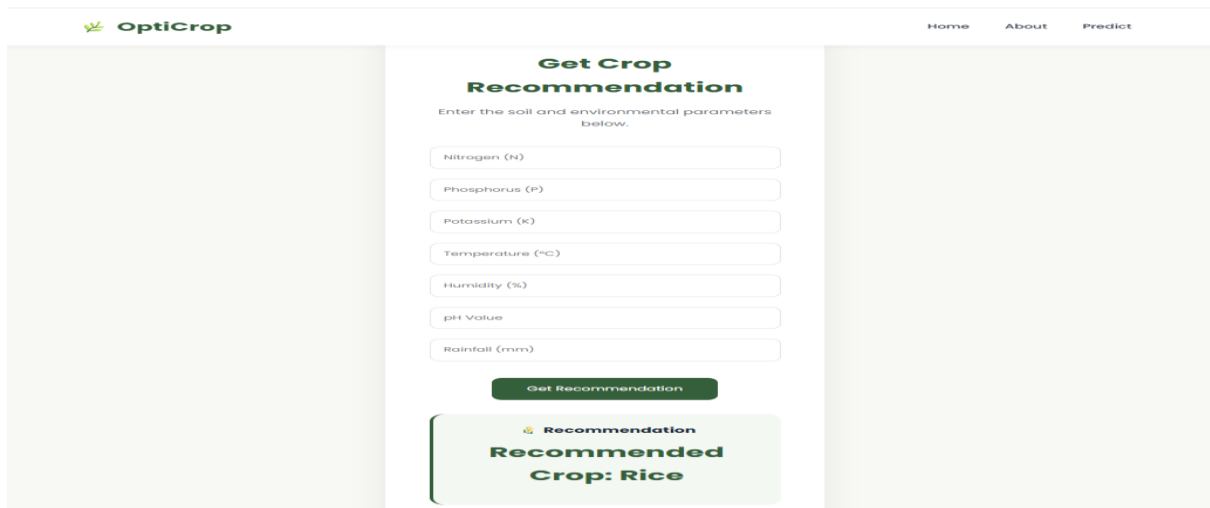
```
In [9]: sample = pd.DataFrame(  
        [[100, 35, 40, 25, 64, 7, 160]],  
        columns=["N", "P", "K", "temperature", "humidity", "ph", "rainfall"]  
    )  
  
    prediction = model.predict(sample)  
  
    print("The suggested crop for given climatic condition is:", prediction)  
  
The suggested crop for given climatic condition is: ['coffee']
```

The prediction process is fast, accurate, and supports informed agricultural decision-making.

Activity 4.4: Result Display & Visualization

The predicted crop is displayed clearly on the output screen along with the user-provided input values. The application presents the recommendation in a simple and easy-to-understand format, allowing users to interpret the results quickly.

Additionally, visualizations such as graphs or charts can be used to analyze crop distribution and agricultural data, making the prediction results more informative and user-friendly.



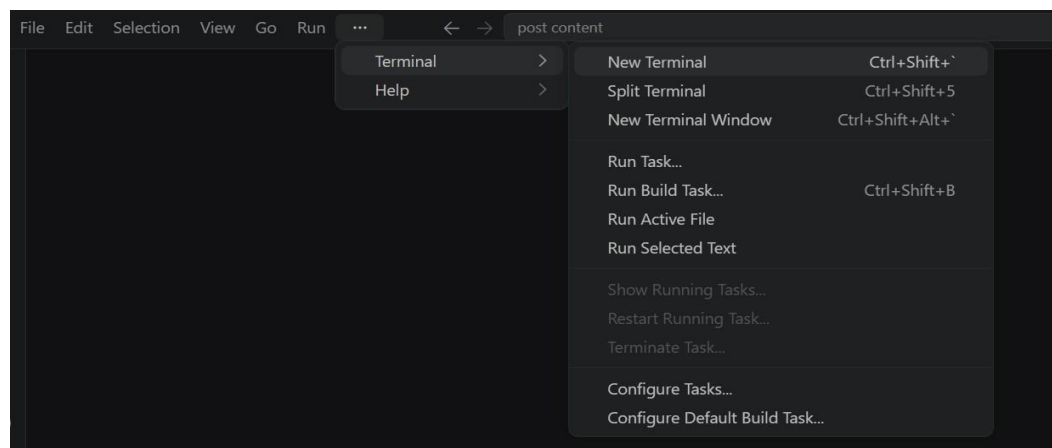
The screenshot displays the OptiCrop web application. At the top left is the logo 'OptiCrop' with a green leaf icon. To the right are navigation links: 'Home', 'About', and 'Predict'. The main heading is 'Get Crop Recommendation', followed by the instruction 'Enter the soil and environmental parameters below.' Below this are seven input fields: 'Nitrogen (N)', 'Phosphorus (P)', 'Potassium (K)', 'Temperature (°C)', 'Humidity (%)', 'pH Value', and 'Rainfall (mm)'. A green 'Get Recommendation' button is positioned below the fields. The result is shown in a green box with the heading 'Recommendation' and the text 'Recommended Crop: Rice'.

Milestone 5: Testing & Deployment

The fifth milestone focused on testing the crop recommendation system to ensure accurate predictions, evaluating the performance of the trained Machine Learning model, and maintaining the project using GitHub for version control and documentation. This milestone ensured that the application was reliable, efficient, and ready for future enhancements.

Activity 5.1: Functional Testing

- Open a New Terminal



Activity 5.2: Performance Evaluation

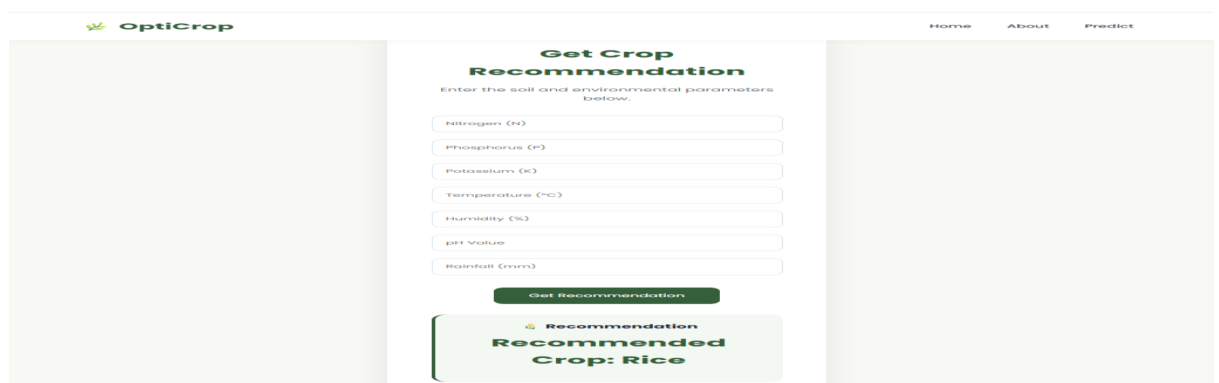
Start the Flask Development Server:

- Run the application locally using:

```
(myenv) D:\projects>python app.py
```

- Once running, open a browser and navigate to “http://127.0.0.1:5050” to access the app.

```
(venv) D:\projectsSB\AI in healthcare\startup-validator>python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment.
d. Follow link (ctrl + click)
* Running on http://127.0.0.1:5050
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 442-110-195
```



The screenshot shows the OptiCrop web application. The header includes the logo and navigation links for Home, About, and Predict. The main section is titled "Get Crop Recommendation" and prompts the user to "Enter the soil and environmental parameters below." There are seven input fields: Nitrogen (N), Phosphorus (P), Potassium (K), Temperature (°C), Humidity (%), pH Value, and Rainfall (mm). A green "Get Recommendation" button is located below the fields. The result is displayed in a green box: "Recommendation Recommended Crop: Rice".

Activity 5.3: GitHub Repository & Project Documentation

The complete project was maintained in a GitHub repository to facilitate version control, collaboration, and project management. All source code, datasets, documentation, and related project files were organized and stored in the repository.

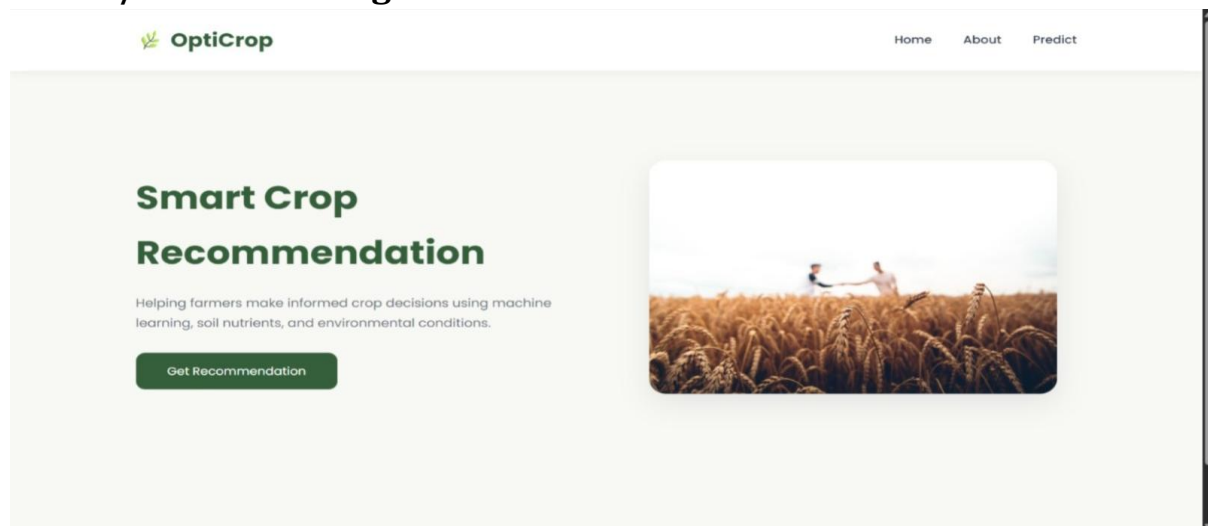
The project documentation includes:

- Project Description
- Problem Statement
- System Architecture
- Data Preprocessing
- Machine Learning Model Development
- Testing Results
- User Guide
- Future Enhancements

Maintaining the project on GitHub enables easy tracking of code changes, efficient collaboration among team members, and simplified project maintenance.

Exploring the Website's Web Pages:

Home / Generator Page:



Description: The Home Page is the main interface of the OptiCrop (Smart Agricultural Production Optimization Engine) application. It provides users with an easy-to-use interface to access the agricultural recommendation system, allowing them to navigate to the prediction tool ("Find Your Crop") where they can enter the required soil nutrient and environmental details to determine the most suitable crop.

Input Section:

A screenshot of the OptiCrop website's input section. The page has a light beige background. At the top left is the 'OptiCrop' logo with a green leaf icon. At the top right are navigation links: 'Home', 'About', and 'Predict'. The main content area is titled 'Get Crop Recommendation' in bold green text. Below the title is a subtitle: 'Enter the soil and environmental parameters below.' There are seven input fields stacked vertically: 'Nitrogen (N)', 'Phosphorus (P)', 'Potassium (K)', 'Temperature (°C)', 'Humidity (%)', 'pH Value', and 'Rainfall (mm)'. Each field is a white rounded rectangle with a thin border. Below the input fields is a green button labeled 'Get Recommendation'.

OptiCrop Home About Predict

Get Crop Recommendation

Enter the soil and environmental parameters below.

90

42

43

20.87

82.00

6.5

202.93

Get Recommendation

DescriptionThe Input Section allows users to enter the soil nutrients and environmental conditions, such as Nitrogen, Phosphorous, Potassium, Temperature, Humidity, pH, and Rainfall. After filling in all the required fields, the user clicks the Predict Crop button to submit the information for prediction.

Results Section:

OptiCrop Home About Predict

Get Crop Recommendation

Enter the soil and environmental parameters below.

Nitrogen (N)

Phosphorus (P)

Potassium (K)

Temperature (°C)

Humidity (%)

pH Value

Rainfall (mm)

Get Recommendation

Recommendation
Recommended Crop: Rice

DescriptionThe Result Section displays the prediction generated by the trained machine learning model. Based on the entered soil nutrients and environmental details, the application shows the recommended crop (such as Rice) in a clear and user-friendly format at the bottom of the form.

Conclusion:

The OptiCrop: Smart Agricultural Production Optimization Engine successfully demonstrates how Machine Learning can support smart and sustainable agriculture by recommending the most suitable crop based on soil nutrients and environmental conditions. Using Google Colab, Python, and Scikit-learn, the system was developed to analyze parameters such as Nitrogen (N), Phosphorus (P), Potassium (K), temperature, humidity, pH, and rainfall to generate accurate crop recommendations. The project includes data preprocessing, model training, evaluation, and prediction, providing farmers with a simple and reliable decision-support tool. Overall, OptiCrop helps improve crop productivity, optimize resource utilization, and contributes to data-driven and sustainable farming practices..